Amazon Seller Central SP-API Setup Guide

This guide walks you through setting up Amazon Selling Partner API (SP-API) access for your seller account.

Overview

The Amazon SP-API wrapper allows you to manage your seller account through natural language commands, including:

- Inventory reorder optimization with cost-benefit analysis
- FBA fee calculations and storage cost projections
- Review monitoring and compliance flagging
- Buy box tracking
- Multi-marketplace operations
- Price optimization

Prerequisites

1. **Amazon Seller Central Account** - Professional selling plan required
2. **AWS Account** - For IAM roles and credentials
3. **Developer Account** - Register as an SP-API developer

Important Cost Information (2026)

Starting in 2026, Amazon charges for SP-API usage:

- **Annual Subscription**: $1,400 USD (starts January 31, 2026)
- **GET Call Fees**: Per-call charges (starts April 30, 2026)
- **Free Operations**: POST, PUT, PATCH calls remain free

Our wrapper uses aggressive caching to minimize GET calls and reduce costs.

Setup Steps

Step 1: Register as SP-API Developer

1. Go to Amazon Developer Console

2. Sign in with your Seller Central credentials
3. Navigate to **"Apps & Services"** → **"App Console"**
4. Click **"Add new app client"**
5. Fill in application details:
   - **App name**: "Marceau Solutions Seller Automation"
   - **OAuth Redirect URI**: `https://localhost` (for testing)
   - **IAM ARN**: (will create in Step 2)

Step 2: Create AWS IAM Role

1. Go to [AWS IAM Console]
2. Create a new IAM user:
   - **User name**: `sp-api-user`
   - **Access type**: Programmatic access
3. Save the **Access Key ID** and **Secret Access Key**
4. Create IAM Role:
   - Go to **Roles** → **Create role**
   - **Trusted entity type**: Custom trust policy
   - Use this trust policy:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {
        "AWS": "arn:aws:iam::437568002678:role/Selling-Partner-API-Role"
      },
      "Action": "sts:AssumeRole"
    }
  ]
}
```

1. Attach policy: `AmazonSPAPIFullAccess` (or create custom restrictive policy)
2. Name the role: `SellingPartnerAPIRole`
3. Copy the **Role ARN** (looks like: `arn:aws:iam::123456789012:role/SellingPartnerAPIRole`)

Step 3: Get SP-API Credentials

1. Return to Developer Console
2. Click on your app
3. Note these credentials:
   - **LWA Client ID** (also called App ID)

- **LWA Client Secret**

4. Click **"Authorize"** to get refresh token:
   - Select selling regions (e.g., North America)
   - Authorize the application
   - Save the **Refresh Token** (only shown once!)

Step 4: Get Your Seller/Merchant ID

1. Go to Seller Central
2. Navigate to **Settings → Account Info**
3. Find **Merchant Token** under "Your Merchant Token"
4. This is your Seller ID

Step 5: Configure Environment Variables

1. Copy `.env.example` to `.env`:

```
cp .env.example .env
```

2. Edit `.env` and add your credentials:

```
# Amazon Selling Partner API (SP-API)
AMAZON_REFRESH_TOKEN=Atzr|IwEBIG...  # From Step 3
AMAZON_LWA_APP_ID=amzn1.application-oa2-client.abc123...  # LWA Client ID
AMAZON_LWA_CLIENT_SECRET=abc123def456...  # LWA Client Secret
AMAZON_AWS_ACCESS_KEY=AKIAIOSFODNN7EXAMPLE  # From Step 2
AMAZON_AWS_SECRET_KEY=wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY  # From Step 2
AMAZON_ROLE_ARN=arn:aws:iam::123456789012:role/SellingPartnerAPIRole  # From Step 2

# Amazon Marketplace Configuration
AMAZON_MARKETPLACE_ID=ATVPDKIKX0DER  # US marketplace (see table below)
```

Marketplace IDs

Marketplace	Marketplace ID
United States	ATVPDKIKX0DER
Canada	A2EUQ1WTGCTBG2
Mexico	A1AM78C64UM0Y8
Brazil	A2Q3Y263D00KWC

United Kingdom	A1F83G8C2ARO7P
Germany	A1PA6795UKMFR9
France	A13V1IB3VIYZZH
Spain	A1RKKUPIHCS9HS
Italy	APJ6JRA9NG5V4
Netherlands	A1805IZSGTT6HS
Japan	A1VC38T7YXB528
Australia	A39IBJ37TRP1C6
Singapore	A19VAU5U5O7RUS

Step 6: Install Python Dependencies

```
# Activate your virtual environment
source venv/bin/activate

# Install required packages
pip install python-amazon-sp-api requests python-dotenv
```

Step 7: Test the Connection

```
# Test basic API connection
python execution/amazon_sp_api.py

# Test inventory optimization
python execution/amazon_inventory_optimizer.py --asin B08XYZ123
```

Usage Examples

Natural Language Commands

Once set up, you can use natural language commands:

Inventory Management:

- "Should I reorder ASIN B08XYZ123? Consider storage fees and buy box risk."
- "Show me all ASINs at risk of aged inventory fees"

- "Which products are at risk of running out in the next 14 days?"

Cost Analysis:

- "Calculate total FBA fees for ASIN B08ABC456 this month"
- "Show me storage cost projections for my top 10 ASINs"

Review Monitoring:

- "Show me all 1-star reviews from this month"
- "Flag reviews that violate Amazon policies"

Buy Box Tracking:

- "Am I winning the buy box for ASIN B08DEF789?"

Price Optimization:

- "Optimize pricing for ASIN B08GHI012 to maximize profit while keeping buy box"

Command Line Usage

```
# Inventory optimization with custom parameters
python execution/amazon_inventory_optimizer.py \
  --asin B08XYZ123 \
  --days 30 \
  --lead-time 45 \
  --target-supply 90

# Get inventory summary
python execution/amazon_sp_api.py
```

Features

1. Inventory Reorder Optimization

- Calculates optimal reorder quantities
- Projects storage costs including 2026 aged inventory fees
- Assesses stockout risk and buy box impact
- Provides cost-benefit analysis for over/under ordering

2. Storage Fee Calculator

- Monthly FBA storage fees
- Aged inventory surcharges (12-15 months: $0.30/unit, 15+ months: $0.35/unit)
- Peak season pricing (Oct-Dec vs Jan-Sep)
- Multi-scenario cost comparisons

3. Review Monitoring

- Flags reviews that may violate Amazon policies
- Identifies fulfillment issues vs product issues
- Provides manual removal instructions (API cannot remove reviews)
- Tracks Order Defect Rate (ODR) impact

4. Cost Optimization

- Aggressive caching to minimize GET calls (reduces 2026 fees)
- Notification-based updates instead of polling
- Batch operations where possible
- API usage tracking and reporting

Limitations & Workarounds

Review Management

Limitation: Amazon SP-API does NOT provide access to individual review text or removal endpoints.

Workaround:

- Use Customer Feedback API for aggregate insights
- Flag potentially violating reviews based on criteria
- Manual removal required through Seller Central UI
- Must request removal within 90 days of review submission

API Rate Limits

Limitation: GET calls will incur fees starting April 30, 2026.

Workaround:

- Aggressive caching (default 30-60 minute cache)
- Subscribe to notifications instead of polling
- Batch requests for multiple ASINs
- Track API usage with built-in counter

Troubleshooting

Authentication Errors

Error: `401 Unauthorized` or `403 Forbidden`

Solutions:

1. Verify refresh token hasn't expired
2. Check IAM role ARN is correct
3. Ensure AWS credentials have proper permissions
4. Re-authorize application in Developer Console

Missing Inventory Data

Error: No inventory data returned

Solutions:

1. Check ASIN is valid and exists in your catalog
2. Verify marketplace ID matches your selling region
3. Ensure FBA inventory (not FBM)
4. Try clearing cache: `rm -rf .tmp/amazon_cache`

Rate Limit Errors

Error: `429 Too Many Requests`

Solutions:

1. Increase cache duration
2. Reduce polling frequency
3. Use notification subscriptions instead of GET calls
4. Implement exponential backoff

Security Best Practices

1. **Never commit `.env` file** - Already in `.gitignore`
2. **Rotate credentials regularly** - Every 90 days recommended
3. **Use IAM least privilege** - Only grant necessary permissions
4. **Monitor API usage** - Track costs and unusual activity
5. **Secure refresh token** - Treat like a password

Next Steps

1. Complete SP-API registration
2. Configure AWS IAM role

3. Add credentials to `.env`
4. Test connection
5. Start with inventory optimization
6. Expand to other operations

Resources

- Amazon SP-API Documentation
- Customer Feedback API
- SP-API 2026 Fee Changes
- Amazon FBA Fees 2026
- Python SP-API Library

Support

For issues with:

- **API Setup**: Refer to Amazon Developer Console documentation
- **AWS Configuration**: Check AWS IAM documentation
- **Script Errors**: Check execution logs and verify credentials in `.env`
- **Feature Requests**: Add to docs/sessions/ notes

Last Updated: 2026-01-04